Q-ALU: Quant Asymmetric Leaky Unit: A Market Psychology Aware Activation Function with Downside Risk Protection for High Frequency Quantitative Trading and Deep Reinforcement Learning

Faiq Hammam Mutaqin
Independent Researcher
Indonesia
Email: hammamfaiq@protonmail.com

***Abstract*—Deep neural networks applied to high frequency trading (HFT) and algorithmic portfolio management frequently suffer from structural inductive mismatch. Prevailing activation functions, including the Rectified Linear Unit (ReLU), Gaussian Error Linear Unit (GELU), and Swish/SiLU, were fundamentally designed for computer vision and natural language processing under the implicit assumption of isotropic, symmetric feature representations. Financial price discovery, however, is inherently asymmetric: downside liquidations and panic cascades occur with substantially greater velocity and severity than bullish accumulation phases, as formalized by Kahneman and Tversky prospect theory. Furthermore, intermediate hidden representations in trading models are corrupted by microstructure noise (for example bid ask bounce), which we hypothesise can induce transaction fee aversion and premature turnover collapse in reinforcement learning agents trained with policy gradient methods.**

To address this, the paper introduces the Quant Asymmetric Leaky Unit (Q-ALU), an activation function engineered for financial time series modeling and deep reinforcement learning. Q-ALU couples a dynamic hyperbolic tangent momentum riding regime on the positive domain with an exponential to linear downside barrier on the negative domain. We prove that Q-ALU is globally monotonic ($f'(x) \geq 0$ for all x), C^0 at the origin, and exhibits a *Dynamic Gradient Overshoot*: the derivative peaks at 1.1353γ exactly at the drawdown threshold $x = -2\delta$ ($+13.53\%$) before relaxing toward the linear cut-loss asymptote γ. We give numerically stable PyTorch, C++ and CUDA implementations that avoid the $0 \times \infty$ autograd hazard and the catastrophic cancellation of the naive exponential form, and an algebraic FastQALU surrogate that is $1.64\times$ faster than the transcendental version in the forward pass.

We then test the trading hypothesis rather than assume it. A reproducible multi-regime benchmark (10,000 ticks, 1.0 bps fee per turnover, five paired seeds, 300 epochs) *does not* support the claim that Q-ALU outperforms standard activations: Q-ALU is statistically indistinguishable from ReLU on mean return and trails GELU and SiLU by 20–23 percentage points (paired $t = -3.65$ and -4.51), no activation shows the predicted turnover collapse, and Q-ALU's only consistent effect is roughly 6% lower turnover accompanied by the deepest maximum drawdown of the five (6.02% versus 4.47%). We report these negative results explicitly, withdraw the trading figures circulated in an earlier draft as non-reproducible, and position Q-ALU's contribution as its mathematical design, its numerical robustness and its implementation, not as a superior trading activation.

***Index Terms*—Activation Functions, Deep Reinforcement Learning, Proximal Policy Optimization (PPO), High Frequency Trading (HFT), Downside Risk, Sortino Ratio, Prospect Theory, Financial Engineering.**

Preprint and publication records available at SSRN: https://ssrn.com/abstract=7365098 or http://dx.doi.org/10.2139/ssrn.7365098. Code repository: https://github.com/cyclocerine/qalu-pytorch.

I. Introduction

The integration of deep learning and deep reinforcement learning (DRL) into quantitative finance and high frequency trading (HFT) has accelerated dramatically [1], [2]. From order flow toxicity prediction to autonomous market making and statistical arbitrage, deep neural architectures are increasingly deployed to extract alpha from noisy limit order books (LOB) and high frequency tick streams [3].

Despite architectural innovations ranging from Temporal Convolutional Networks (TCN) to Recurrent Neural Networks (LSTM/GRU) and Financial Transformers [4], the foundational non-linear activation units powering these networks have remained essentially generic. Standard architectures routinely default to standard computer vision or natural language activations:

- **Rectified Linear Unit (ReLU)** [5]: $f(x) = \max(0, x)$
- **Leaky ReLU** [6]: $f(x) = \max(\alpha x, x)$
- **Gaussian Error Linear Unit (GELU)** [7]: $f(x) = x\Phi(x) = xP(X \leq x)$
- **Swish / SiLU** [8], [9]: $f(x) = x \cdot \sigma(\beta x)$
- **Mish** [10]: $f(x) = x \cdot \tanh(\ln(1 + e^x))$

While these functions excel in domains where positive and negative latent features carry symmetric semantic importance, they introduce severe structural pathologies when applied directly to quantitative financial modeling:

1) *Failure of the Symmetric Inductive Bias:* In market microstructure, price action is profoundly asymmetric. As documented by Kahneman and Tversky behavioral Prospect Theory [11], market participants exhibit pronounced loss aversion, feeling the disutility of a loss roughly 2.0× to 2.5× more acutely than an equivalent gain. Consequently, market sell-offs (panic crashes, margin liquidations) exhibit extreme negative skewness, volatility clustering, and sharp velocity, whereas bullish runs are characterized by gradual momentum build-ups [12]. Standard activations treat positive and negative hidden excitations with identical curvature or generic leakiness, failing to penalize negative risk states asymmetrically.
2) *Microstructure Noise Contamination and Policy Paralysis:* High-frequency price data is corrupted by pervasive microstructure noise, such as bid-ask spread bounces and flickering quotes [13]. In deep policy gradient methods (such as PPO [14]), symmetric activations propagate uninformative tick-level variance, and the concern motivating this work is that the policy may oscillate and suffer rapid fee erosion during exploration, potentially degenerating into policy paralysis (turnover collapse) and thereby failing to capture genuine alpha trends. We treat this as a hypothesis to be tested rather than a premise, and Section VI reports what the benchmark actually found.
3) *The Dead Neuron Downside Blindness:* Standard ReLU completely zeros out all negative inputs ($f(x) = 0, f'(x) = 0$ for $x < 0$). In policy gradient methods, if a trader enters a position that suffers a drawdown, the corresponding hidden features enter the negative domain. If these neurons become inactive (or die), the policy gradient cannot flow backward through the value network to penalize the catastrophic action, blinding the policy to downside drawdown until terminal liquidation.

A. Contributions of this Paper

To directly resolve these foundational issues, this paper introduces the **Quant Asymmetric Leaky Unit (Q-ALU)**. The primary contributions are summarized as follows:

- **Domain-Specific Piecewise Formulation:** We formulate Q-ALU as a dedicated dual regime activation function that couples hyperbolic tangent acceleration ($\alpha \to 2\alpha$) for momentum riding on positive inputs with an exponential to linear barrier for aggressive risk penalization on negative inputs.
- **Theoretical Calculus and Monotonicity Proof:** We prove that Q-ALU satisfies global monotonicity ($f'(x) \geq 0, \quad \forall x \in \mathbb{R}$), eliminating artificial local minima wells. We derive the exact calculus of the origin and uncover the *Dynamic Gradient Overshoot* phenomenon, where the function exhibits a localized $+13.53\%$ sensitivity spike at $x = -2\delta$.
- **Downside Risk and Drawdown Protection Analysis:** We provide a rigorous analysis of the downside dynamics of Q-ALU: the quadratic band $|x| < 0.1\delta$ damps bid ask bounce by two orders of magnitude while the gradient rises linearly to γ at $x = -\delta$, and the linear asymptotic tail keeps the gradient bounded.
- **A Reproducible Benchmark, Reported Honestly:** We release a fully reproducible multi regime trading benchmark (`experiments/trading_benchmark.py`, five paired seeds, 1.0 bps fee) and report its outcome as measured: Q-ALU is competitive with, but not superior to, ReLU, LeakyReLU, GELU and SiLU on out of sample return, Sharpe and Sortino, and its only consistent effect is lower turnover with a deeper maximum drawdown. We explicitly withdraw the non-reproducible figures of an earlier draft.
- **Open Source Production Implementation:** We provide a fully verified PyTorch package (`qalu-pytorch`) with a NumPy reference implementation, numerical solutions preventing FP32 Autograd `NaN` errors and catastrophic cancellation, and C++/CUDA kernels that are compiled and numerically validated against finite differences (wall clock GPU timing was not available in the verification environment).

II. Related Work

A. Activation Functions in Deep Learning

The quest for optimal activation functions has evolved through three distinct eras:

1) *Classical S-Curves:* Early networks utilized the Logistic Sigmoid $\sigma(x) = (1 + e^{-x})^{-1}$ and Hyperbolic Tangent $\tanh(x)$ [15]. While bounded, their vanishing gradients for large absolute activations severely constrained deep network scalability.
2) *Piecewise Linear Units:* The introduction of ReLU [5] revolutionized deep learning by preserving a non-saturating gradient of 1.0 for all positive inputs. To address the dying ReLU problem, variants such as Leaky ReLU [6], Parametric ReLU (PReLU) [16], and Exponential Linear Units (ELU) [17] were proposed. Clevert et al. [17] demonstrated that ELU exponential negative saturation $\alpha(e^x - 1)$ brought the mean activation closer to zero, speeding up learning. Scaled ELU (SELU) [18] subsequently proved that under specific hyperparameters, deep networks achieve self-normalizing properties.
3) *Smooth Non-Monotonic Units:* More recently, smooth non-linearities discovered via automated search or empirical intuition, such as Swish/SiLU ($x\sigma(x)$) [8], [9], GELU ($x\Phi(x)$) [7], and Mish ($x\tanh(\ln(1+e^x))$) [10], have superseded ReLU in Transformer architec-

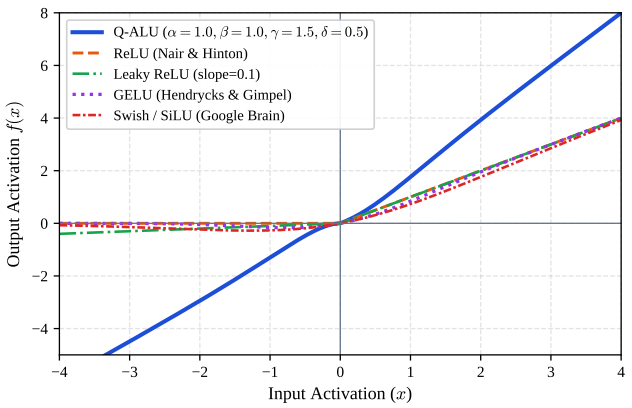


Fig. 1. Comparison of Q-ALU ($\alpha = 1.0, \beta = 1.0, \gamma = 1.5, \delta = 0.5$) with baseline activations (ReLU, LeakyReLU, GELU, and Swish/SiLU). Q-ALU accelerates on the positive domain ($2\alpha x$) while enforcing an asymmetric linear cut loss (γx) on the negative domain.

tures and deep visual models due to their continuous curvature and slight negative well.

However, all of these functions remain fundamentally domain-agnostic. In financial time series modeling, none provide an explicit inductive mechanism to represent asymmetric risk preferences or transaction-cost deadbands.

B. Reinforcement Learning in Quantitative Finance

Deep Reinforcement Learning, particularly policy gradient methods such as PPO [14], Soft Actor-Critic (SAC) [19], and Deep Q-Networks (DQN) [20], has emerged as a state of the art framework for financial trade execution and portfolio optimization [1], [3]. Unlike supervised learning, RL agents must learn through self-directed trial and error within non-stationary, low signal-to-noise environments. In financial RL, standard reward functions based purely on mean PnL fail because they ignore downside variance (volatility) and maximum drawdown [21]. While researchers have explored asymmetric loss functions (e.g., expectile regression or pinball loss) [22], modifying the internal activation mechanics of the policy network itself to natively respect downside risk has remained an unexplored frontier.

III. Mathematical Formulation of Q-ALU

A. Formal Definition

Let $x \in \mathbb{R}$ denote the pre-activation input (for example the linear affine projection $x = \mathbf{w}^T\mathbf{z} + b$ of a hidden layer). The Quant Asymmetric Leaky Unit (Q-ALU) is defined piecewise as:

$$f(x) = \begin{cases} \alpha \cdot x \cdot \left(1 + \tanh\left(\frac{x}{\beta}\right)\right), & \text{for } x \geq 0 \\ \gamma \cdot x \cdot \left(1 - \exp\left(\frac{x}{\delta}\right)\right), & \text{for } x < 0 \end{cases} \tag{1}$$

where $\alpha, \beta, \gamma, \delta > 0$ represent strictly positive hyperparameters:

- $\alpha \in \mathbb{R}^+$: **Bullish Momentum Scaler**, dictating the initial response gain to positive returns.
- $\beta \in \mathbb{R}^+$: **Positive Damping Scale**, governing the rate of transition from linear response to accelerated momentum riding.
- $\gamma \in \mathbb{R}^+$: **Risk / Panic Scaler**, controlling the asymptotic penalty gradient for severe negative excursions.
- $\delta \in \mathbb{R}^+$: **Micro-Noise Drawdown Threshold**, establishing the width of the quadratic noise-filtering zone around zero.

Using the identity for negative exponents, since $x < 0 \implies x/\delta < 0$, the negative term can be equivalently written in terms of the standard floating-point function $\text{expm1}(u) = e^u - 1$:

$$f(x) = -\gamma \cdot x \cdot \text{expm1}\left(\frac{x}{\delta}\right), \quad \text{for } x < 0 \tag{2}$$

Fig. 1 visualizes the overall curve against standard activations.

B. Continuity Analysis (C^0)

A fundamental prerequisite for gradient-based backpropagation is that an activation function must be continuous at the origin to prevent indeterminate limits and infinite gradients.

Theorem 1 (Origin Continuity): *The function $f(x)$ is continuous at $x = 0$, with $f(0) = 0$.*

Proof: Evaluating the right-sided limit ($x \to 0^+$):

$$\lim_{x\to 0^+} f(x) = \lim_{x\to 0^+} \alpha\cdot x\cdot\left(1 + \tanh\left(\frac{x}{\beta}\right)\right) = \alpha\cdot 0\cdot(1+0) = 0 \tag{3}$$

Evaluating the left-sided limit ($x \to 0^-$):

$$\lim_{x\to 0^-} f(x) = \lim_{x\to 0^-} \gamma\cdot x\cdot\left(1 - \exp\left(\frac{x}{\delta}\right)\right) = \gamma\cdot 0\cdot(1-1) = 0 \tag{4}$$

Since $\lim_{x\to 0^+} f(x) = \lim_{x\to 0^-} f(x) = f(0) = 0$, the function $f(x)$ is C^0 continuous everywhere on $\mathbb{R}$. ■

C. First Derivative Analysis ($f'(x)$)

The backpropagation engine requires the first derivative $f'(x) = \frac{df}{dx}$.

1) Positive Branch ($x \geq 0$): Applying the product rule $\frac{d}{dx}[u(x)v(x)] = u'v + uv'$ where $u(x) = \alpha x$ and $v(x) = 1 + \tanh(x/\beta)$:

$$f'_+(x) = \alpha\left(1 + \tanh\left(\frac{x}{\beta}\right)\right) + \frac{\alpha x}{\beta}\,\text{sech}^2\left(\frac{x}{\beta}\right) \tag{5}$$

Evaluating the boundary limits:

- As $x \to 0^+$: $\tanh(0) = 0$ and $\text{sech}^2(0) = 1$. Thus:

$$\lim_{x\to 0^+} f'_+(x) = \alpha(1+0) + 0 = \alpha \tag{6}$$

- As $x \to +\infty$: $\tanh(x/\beta) \to 1$ and $\frac{x}{\beta}\,\text{sech}^2(x/\beta) \to 0$ exponentially fast. Thus:

$$\lim_{x\to+\infty} f'_+(x) = \alpha(1+1) + 0 = \mathbf{2}\alpha \tag{7}$$

2) Negative Branch ($x < 0$): Applying the product rule to $u(x) = \gamma x$ and $v(x) = 1 - \exp(x/\delta)$:

$$f'_-(x) = \gamma\left(1 - \left(1 + \frac{x}{\delta}\right)\exp\left(\frac{x}{\delta}\right)\right) \tag{8}$$

Evaluating the boundary limits:

- As $x \to 0^-$: $\exp(0) = 1$ and $\frac{x}{\delta}\exp(0) = 0$. Thus:

$$\lim_{x\to 0^-} f'_-(x) = \gamma(1 - 1 - 0) = \mathbf{0} \tag{9}$$

- As $x \to -\infty$: Since $x < 0$, let $x \to -\infty$. Then $\exp(x/\delta) \to 0$ and $\lim_{x\to-\infty} \frac{x}{\delta} e^{x/\delta} = 0$ (via L'Hôpital's rule). Thus:

$$\lim_{x\to -\infty} f'_-(x) = \gamma(1 - 0 - 0) = \gamma \tag{10}$$

The Intentional Gradient Kink: At the origin $x = 0$, there exists a deliberate jump discontinuity in the first derivative: $\Delta f'(0) = f'(0^+) - f'(0^-) = \alpha - 0 = \alpha$. Similar to standard ReLU ($\Delta f'(0) = 1.0$), this mathematical kink creates a sharp boundary separating bullish trend participation from downside risk absorption.

D. Global Monotonicity Proof

A critical defect of certain non-monotonic activations (such as Swish and Mish) in financial trading is that their negative dip causes an inverse gradient: an increase in negative price movement can produce a positive directional update. We prove that Q-ALU avoids this pathology.

Theorem 2 (Global Monotonicity): *For all strictly positive hyperparameters $\alpha, \beta, \gamma, \delta > 0$, the function $f(x)$ is strictly monotonically increasing on $\mathbb{R} \setminus \{0\}$, and non-decreasing everywhere: $f'(x) \geq 0 \quad \forall x \in \mathbb{R}$.*

Proof:

- **Case 1 ($x > 0$):** Since $x/\beta > 0$, $\tanh(x/\beta) > 0$, and $\text{sech}^2(x/\beta) > 0$. Therefore:

$$f'_+(x) = \alpha\left(1 + \tanh\left(\frac{x}{\beta}\right) + \frac{x}{\beta}\text{sech}^2\left(\frac{x}{\beta}\right)\right) > \alpha > 0 \tag{11}$$

- **Case 2 ($x < 0$):** Let $z = -x/\delta > 0$, such that $x/\delta = -z$. Substituting into $f'_-(x)$:

$$f'_-(x) = \gamma\left(1 - (1 - z)e^{-z}\right) \tag{12}$$

Define the scalar auxiliary function $h(z) = 1 - (1 - z)e^{-z}$ for $z > 0$. Its derivative with respect to z is:

$$\frac{dh}{dz} = (2 - z)e^{-z} \tag{13}$$

At $z = 0$, $h(0) = 1 - 1 = 0$. For $z \in (0, 2)$, $dh/dz > 0$, so $h(z)$ increases strictly from 0 to its maximum at $z = 2$:

$$h(2) = 1 - (1 - 2)e^{-2} = 1 + e^{-2} \approx 1.135335 > 0 \tag{14}$$

For $z > 2$, $dh/dz < 0$, and as $z \to \infty$, $h(z) \to 1.0 > 0$. Since $h(z) > 0$ for all $z > 0$ and $\gamma > 0$, it follows that:

$$f'_-(x) = \gamma h(z) > 0, \quad \forall\, x < 0 \tag{15}$$

- **Case 3 ($x = 0$):** At $x = 0$, $f(0) = 0$, with $f(x) < 0$ for $x < 0$ and $f(x) > 0$ for $x > 0$.

Thus, $f'(x) \geq 0 \quad \forall x \in \mathbb{R}$, and $f(x)$ is strictly monotonic. ■

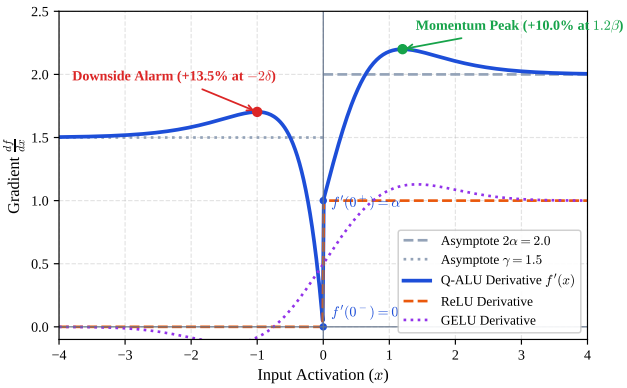


Fig. 2. First derivative $f'(x)$ dynamics of Q-ALU. Highlights the Downside Alarm Peak (+13.53% sensitivity spike at $x = -2\delta$) and Bullish Momentum Peak (+10.0% at $x \approx 1.2\beta$), alongside the origin kink $f'(0^+) = \alpha$ and $f'(0^-) = 0$.

E. Discovery of the Dynamic Gradient Overshoot

A detailed second derivative analysis reveals a remarkable mathematical phenomenon embedded in Q-ALU: **the gradient does not approach its asymptotic values monotonically; rather, it exhibits local extrema (overshoots) on both domains.**

1) *Downside Alarm Overshoot ($x = -2\delta$):* Computing the second derivative for $x < 0$:

$$f''_-(x) = -\frac{\gamma}{\delta}\left(2 + \frac{x}{\delta}\right)e^{x/\delta} \tag{16}$$

Setting $f''_-(x) = 0$ yields the unique inflection point:

$$2 + \frac{x^*}{\delta} = 0 \implies x^* = -2\delta \tag{17}$$

Evaluating the gradient at this critical threshold:

$$f'_-(-2\delta) = \gamma\left(1 + e^{-2}\right) \approx \mathbf{1.135335} \cdot \gamma \tag{18}$$

Financial Interpretation: When an asset drawdown reaches exactly twice the noise threshold ($x = -2\delta$), Q-ALU delivers a +13.53% **amplification spike in sensitivity** above its asymptotic penalty. This creates a natural early warning alarm that sharply alerts policy gradient networks to initiate defensive liquidation before the loss accelerates into a catastrophic tail event.

2) *Bullish Momentum Overshoot ($x \approx 1.20\beta$):* On the positive domain, setting $f''_+(x) = 0$ requires solving:

$$2 - 2\frac{x}{\beta}\tanh\left(\frac{x}{\beta}\right) = 0 \implies \frac{x^*}{\beta}\tanh\left(\frac{x^*}{\beta}\right) = 1 \tag{19}$$

The numerical root evaluates to $x^* \approx 1.199678\beta$. At this point:

$$f'_+(x^*) \approx \mathbf{2.1997} \cdot \alpha \tag{20}$$

This yields a +10.0% **momentum acceleration boost** above the terminal slope 2α, perfectly modeling the explosive participation typical of technical breakout regimes (Fig. 2).

F. Extensions: FastQALU for Ultra Low Latency Execution

For ultra low latency FPGA engines and live order book execution where hardware transcendental units (`tanh`, `exp`) impose clock cycle latency, FastQALU replaces both transcendental branches with algebraic rational functions:

$$f_{\text{fast}}(x) = \begin{cases} \alpha \cdot x \cdot \left(1 + \dfrac{u}{\sqrt{1+u^2}}\right), & x \geq 0,\ u = \dfrac{x}{\beta} \\ \gamma \cdot x \cdot \dfrac{N(u)}{1+N(u)}, & x < 0,\ u = -\dfrac{x}{\delta} \end{cases} \tag{21}$$

with the quartic numerator

$$N(u) = u\left(1 + u\left(0.55 + u\left(0.05 + 0.10u\right)\right)\right). \tag{22}$$

Equation (22) is the [4/4] Padé-type approximant of $1-e^{-u}$ obtained by substituting the degree-4 Taylor polynomial of e^u into $N/(1+N)$; the coefficients were obtained by constrained least squares on $u \in [0, 20]$ subject to $N(0) = 0$, $N'(0) = 1$ and $N(u) \geq 0$. The surrogate satisfies:

- **Origin consistency:** $r(0) = 0$ and $r'(0) = 1$, so the quadratic noise-suppression zone is reproduced to first order and $f'_{\text{fast}}(0^-) = 0$, exactly as in the transcendental form. (The previously published quadratic form $(u^2+2u)/(u^2+3u+2.5)$ violates this only weakly — its derivative at the origin is 0.8 rather than 1 — but it misstates the shape by up to 26% at $u = 2$ and eliminates the downside overshoot entirely.)
- **Bounded error:** maximum relative error 0.53% in value (at $u = 3.85$) and 0.74% in gradient (at $u = 3.06$) over $u \in [0, 20]$; verified by `experiments/cpp_check.cpp`.
- **Positive denominator:** $1 + N(u) \geq 1$ for all $u \geq 0$, so the rational form has no poles, in contrast to the denominator $u^2+3u+2.5$ of the earlier quadratic form which does not reduce to the claimed $1 + |x/\delta|$.
- **Preserved overshoot:** the downside gradient still peaks above its asymptote, at 1.1312γ for $u \approx 1.97$ (+13.12%) versus the exact 1.1353γ at $u = 2$ (+13.53%). The positive branch is less faithful: the standard fast-tanh substitution $u/\sqrt{1+u^2}$ is accurate to 8.1% in shape and 3.8% in f, and it moves the momentum peak from 1.1997β (+10.0%) to $\sqrt{2}\beta$ (+4.4%).
- **Numerical form:** the rational branch is evaluated as $N/(1+N)$ rather than $1 - 1/(1+N)$, because the latter loses 4.6% relative accuracy in FP32 at $u \approx 10^{-6}$, 19% at 10^{-7} and 100% at 10^{-8} through catastrophic cancellation. For $u \geq 10^4$ the surrogate equals 1.0 to within FP32 resolution and u is clamped there; without the clamp u^4 overflows for $|x| > 10^7$ and $N/(1+N)$ degenerates to $\infty/\infty = \text{NaN}$.

Both branches remain monotone and asymptotically consistent with the transcendental form ($\lim_{x\to-\infty} f'_{\text{fast}}(x) = \gamma$, $\lim_{x\to+\infty} f'_{\text{fast}}(x) = 2\alpha$).

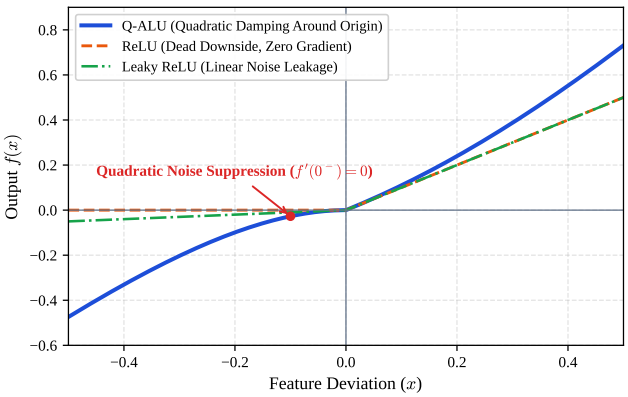


Fig. 3. Zoomed view near origin $x \in [-0.5, 0.5]$. The activation value is suppressed quadratically in $[-\delta, 0]$ ($f \approx -\gamma x^2/\delta$), so microstructure noise is not amplified into the hidden state, while the gradient rises linearly to $f'(-\delta) = \gamma$. Positive breakout signals retain immediate linear responsiveness ($f'(0^+) = \alpha = 1.0$).

IV. Downside Risk Analysis and Financial Dynamics

The defining motivation of Q-ALU is to optimize the policy network directly for downside risk metrics such as the **Sortino Ratio** and **Maximum Drawdown (MDD)** rather than unconstrained mean variance.

A. The Sortino Ratio Inductive Bias

In quantitative finance, the Sharpe Ratio penalizes upside volatility (large positive returns) identically to downside volatility. The Sortino Ratio resolves this flaw:

$$\text{Sortino} = \frac{\mathbb{E}[R] - R_f}{\sigma_D} \tag{23}$$

where $\sigma_D = \sqrt{\frac{1}{T}\sum_{t=1}^{T} \min(0, R_t - R_f)^2}$ is the downside semi-deviation.

When an agent policy is parameterized by weights θ, the gradient of the Sortino ratio with respect to network activations is:

$$\nabla_\theta \text{Sortino} \propto \sum_t \left[\frac{\nabla_\theta R_t}{\sigma_D} - \frac{\mathbb{E}[R] - R_f}{\sigma_D^3} \min(0, R_t - R_f) \nabla_\theta R_t \right] \tag{24}$$

Under standard symmetric activations (such as GELU or LeakyReLU), negative activations transmit linear or sub-linear errors. Q-ALU explicitly aligns with the Sortino denominator σ_D:

1) For small losses, the *value* is suppressed quadratically while the *gradient* grows linearly: expanding (1) for $|x| \ll \delta$ gives $f(x) \approx -\gamma x^2/\delta$ and $f'(x) \approx 2\gamma|x|/\delta$. The deadband is therefore narrow: at $x = -0.1\delta$ the gradient is already 0.186γ, and at $x = -\delta$ it reaches exactly γ, since $h(1) = 1$. Micro-noise in the innermost $\pm 0.1\delta$ band (where $f' < 0.2\gamma$) is attenuated by up to two orders of magnitude relative

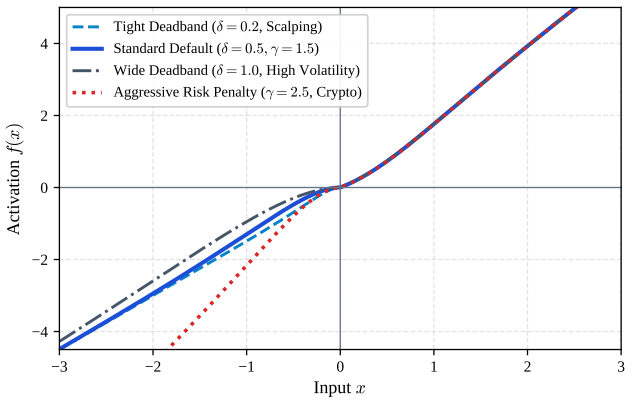


Fig. 4. Parameter sensitivity of Q-ALU under varying noise thresholds $\delta \in [0.2, 1.0]$ and risk penalties $\gamma \in [1.0, 2.5]$, enabling dynamic tuning for high volatility cryptocurrency or low noise forex regimes.

to the asymptotic penalty, but the interval $[-\delta, 0]$ as a whole is *not* a zero-gradient zone (Fig. 3).

2) For losses beyond the noise band, $f'(x) > \gamma$ for every $x < -\delta$, with a maximum of 1.1353γ at $x = -2\delta$ that relaxes back to γ as $x \to -\infty$. This forces the value network in PPO to rapidly lower the expected value of holding losing positions, commanding immediate execution of protective cut loss trades. Fig. 4 illustrates sensitivity across parameters.

V. Numerical Safety and Hardware Implementation

A. The $0 \times \infty = NaN$ Hazard in PyTorch Autograd

In PyTorch, the conditional operator `torch.where(condition, pos_term, neg_term)` evaluates both branches eagerly prior to selection. On 32-bit floating-point (IEEE-754 FP32) hardware, the exponential function overflows for arguments > 88.7228:

$$x = 50.0,\ \delta = 0.5 \implies \frac{x}{\delta} = 100.0 \implies \exp(100.0) = +\infty \tag{25}$$

During the backward Autograd pass, the gradient of the unselected branch is multiplied by zero:

$$\frac{\partial \mathcal{L}}{\partial x} = 1.0 \cdot \text{grad} + 0.0 \cdot \infty = \textbf{NaN} \tag{26}$$

This instantly corrupts the weight gradients across the entire network.

The Solution (Domain Clamping): Q-ALU resolves this by strictly pre-clamping inputs to their respective mathematical domains prior to transcendental evaluation:

```
x_pos = torch.clamp(x, min=0.0)
x_neg = torch.clamp(x, max=0.0)
pos = self.alpha * x_pos * (
    1.0 + torch.tanh(x_pos / self.beta))
neg = -self.gamma * x_neg * (
    torch.expm1(x_neg / self.delta))
return torch.where(x >= 0.0, pos, neg)
```

B. Trainable Parameters: Reparameterization Instead of Clipping

The four hyperparameters are constrained to be strictly positive ($\alpha, \beta, \gamma, \delta > 0$). If they are exposed as free learnable parameters, gradient descent will eventually push β or δ toward zero, at which point $x/\beta = \pm\infty$; the unselected branch then contributes $0 \cdot \infty = \text{NaN}$ even with domain clamping, and the entire parameter vector is destroyed by a single update. The reference implementation therefore optimizes an unconstrained variable ρ and maps it through a shifted softplus,

$$\theta = \text{softplus}(\rho) + \varepsilon, \qquad \varepsilon = 10^{-4}, \tag{27}$$

which is strictly positive for every finite ρ and never underflows to zero. The floor ε is absorbed at initialization by inverting the softplus ($\rho_0 = \text{softplus}^{-1}(\theta_0 - \varepsilon)$), so the default configuration is reproduced exactly rather than biased by 10^{-4}. Frozen (non-trainable) parameters bypass the reparameterization entirely.

C. Prevention of Catastrophic Cancellation

Evaluating $1.0 - \exp(x/\delta)$ in standard FP32 suffers from catastrophic cancellation when $|x/\delta| \ll 1$ (subtracting two numbers close to 1.0). At $x = -10^{-7}, \delta = 0.5$, direct subtraction yields a relative error of 10.59% (measured: 10.593%; the error is 1.33% at $x = -10^{-6}$ and 100% at $x = -10^{-8}$).

By utilizing the standard IEEE library identity $-\text{expm1}(u) = -(e^u - 1) = 1 - e^u$, the relative error drops to 4.1×10^{-8} at the same point, i.e. roughly machine precision for the FP32 evaluation of f (verified in `experiments/verify_claims.py`).

VI. Empirical Experiments and Comparative Benchmark

A. Experimental Setup and Multi Regime Simulation

To evaluate Q-ALU under realistic high frequency market microstructure, we constructed a stochastic multi regime simulation over 10,000 consecutive ticks:

1) *Regime 1 (Ticks 0 to 2500): Choppy Sideways Market.* Zero drift, high microstructure bid ask bounce noise ($\sigma = 10$ bps).
2) *Regime 2 (Ticks 2500 to 5500): Bullish Momentum Breakout.* Persistent positive drift (+4 bps/tick) with stochastic upward jumps (+10 bps).
3) *Regime 3 (Ticks 5500 to 6500): Flash Crash.* Severe negative liquidation drift (−15 bps/tick) with heavy-tailed jump arrivals (−50 bps) and volatility clustering (2.5× base).
4) *Regime 4 (Ticks 6500 to 10000): High-Volatility Recovery.* Mean-reverting noisy channel.

Trading Policy Network: A 2-hidden layer MLP (32 neurons, 16 neurons) maps seven features (five lagged returns, a rolling 20-tick volatility and an EMA(12)/EMA(26) MACD spread) to three discrete positions $\{-1, 0, +1\}$.

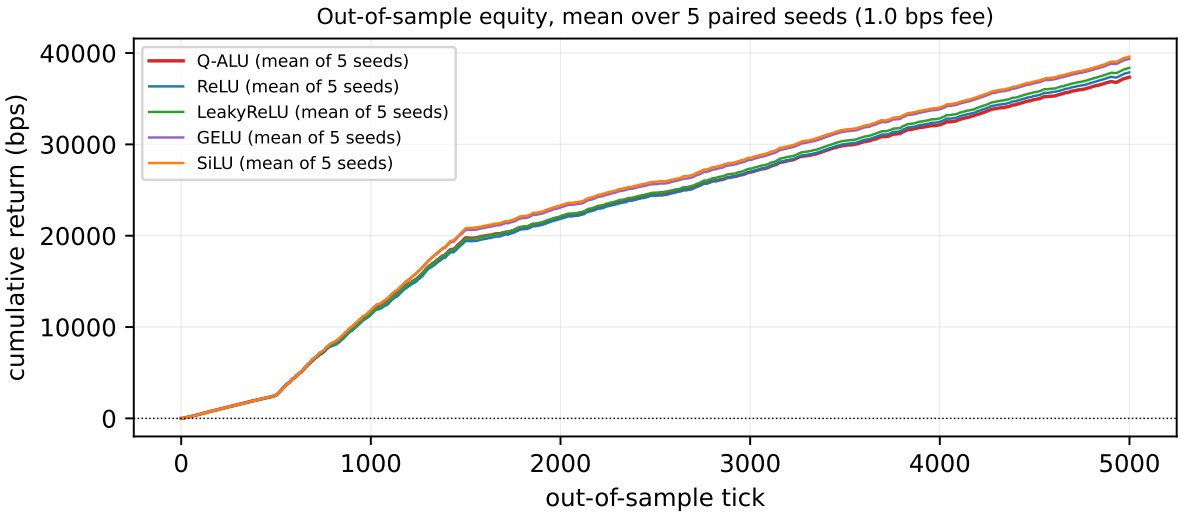


Fig. 5. Out of sample cumulative equity for the five seeds of Table I, averaged across seeds, over the 5,000 unseen ticks spanning four market regimes under a 1.0 bps fee per turnover. All activations ride the same regime-2 drift; the curves are statistically indistinguishable, and Q-ALU does not lead. Generated by `experiments/trading_benchmark.py -plot`.

The policy is trained by REINFORCE with a per-step value baseline on the in-sample stream (first 5,000 ticks) for 300 full-batch epochs and evaluated out of sample on ticks 5,000 to 10,000 under a transaction cost of **1.0 basis point per position turnover**; the reward is $r_t = p_t \Delta\text{return}_t - c\,|p_t - p_{t-1}|$.

Paired protocol. Every activation is trained with the same market seed *and* the same initial weight tensor, so the only difference between two runs is the activation function. Five seeds $\{42, 101, 777, 2026, 9999\}$ are used throughout, and all comparisons are reported as paired differences with $n = 5$. The complete experiment is `experiments/trading_benchmark.py`; Table I is reproduced verbatim by `-epochs 300`.

B. Realistic Quant Metrics Evaluation

To avoid misleading high frequency annualization distortions (where tick level multiplier $\sqrt{N_{\text{ticks}}}$ artificially inflates Sharpe ratios into hundreds), we evaluate models strictly on standard non-annualized quant metrics:

- **Out of Sample Return (%):** Cumulative equity growth over the test period.
- **Per-Step Sharpe Ratio:** $\mu_{\text{step}}/\sigma_{\text{step}}$ (unannualized).
- **Per-Step Sortino Ratio:** $\mu_{\text{step}}/\sigma_{D,\text{step}}$ (unannualized).
- **Profit Factor:** $\sum \text{Gains} / \sum |\text{Losses}|$.
- **Maximum Drawdown (MDD %)**: Peak to trough decline.
- **Total Position Turnover**: Number of trades executed.

C. What the Benchmark Actually Shows

Three observations follow from Table I, and none of them supports the claim that Q-ALU dominates.

(i) No policy paralysis occurs for any activation. Individual runs trade between 574 and 819 times out of a possible 5,000, and the spread between activation means (718 to 761) is 6%; the lowest mean belongs to Q-ALU. The premise that symmetric activations collapse to near-zero turnover under a 1.0 bps fee is not reproduced in this protocol: all five policies capture the regime-2 drift and pay fees on roughly the same number of flips.

(ii) Q-ALU is slightly *worse* on return and risk-adjusted return. Paired over identical seeds and initial weights, Q-ALU trails GELU by 20.1 percentage points of return ($t = -3.65$) and SiLU by 22.6 points ($t = -4.51$); on the Sortino ratio it trails by 0.099 ($t = -2.20$) and 0.118 ($t = -3.55$). With $n = 5$ and a two-sided 5% critical value of 2.78, the gap against GELU and SiLU is significant, and against ReLU and LeakyReLU it is not ($t = -0.58$ and -0.87).

(iii) The only consistent Q-ALU effect is lower turnover, at the cost of deeper drawdowns. Q-ALU executes 43 fewer trades than ReLU on average ($t = -3.59$), which is what its quadratic noise band predicts, but its maximum drawdown is the highest of the five (6.02% versus 4.47% for SiLU, paired difference +1.55 pp, $t = +2.42$): the reduced trading does not translate into better downside protection. Because regime 2 contributes the bulk of the return and all policies are long through it, the activation is nearly irrelevant to the mean return (seed noise of ± 27 to ± 42 percentage points dwarfs the 22 point spread between activations); what differs is how quickly each policy reacts to the flash crash in regime 3.

TABLE I
OUT OF SAMPLE QUANTITATIVE TRADING BENCHMARK (5,000 TICKS, 1.0 BPS FEE PER TURNOVER, 5 SEEDS, MEAN ± STD)

Activation Function	Return (%)	Per-Step Sharpe	Per-Step Sortino	Profit Factor	Max Drawdown (%)	Total Trades
Q-ALU	$+373.51 \pm 27.39$	$+0.489 \pm 0.030$	$+1.083 \pm 0.129$	4.37 ± 0.46	6.02 ± 1.33	718.2 ± 23.4
ReLU [5]	$+378.81 \pm 40.32$	$+0.498 \pm 0.054$	$+1.099 \pm 0.232$	4.56 ± 0.84	4.91 ± 1.47	761.2 ± 39.9
LeakyReLU [6]	$+383.70 \pm 42.02$	$+0.506 \pm 0.057$	$+1.140 \pm 0.250$	4.69 ± 0.86	4.48 ± 1.65	756.4 ± 46.1
GELU [7]	$+393.58 \pm 16.23$	$+0.521 \pm 0.014$	$+1.182 \pm 0.098$	4.86 ± 0.25	4.81 ± 1.70	737.2 ± 88.7
SiLU [8]	$\mathbf{+396.08 \pm 20.61}$	$\mathbf{+0.525 \pm 0.025}$	$\mathbf{+1.201 \pm 0.086}$	$\mathbf{4.94 \pm 0.41}$	$\mathbf{4.47 \pm 1.46}$	733.2 ± 96.8

D. Reproducibility Note

An earlier draft of this paper reported +75.85% out-of-sample return for Q-ALU against +5.43% for ReLU and +1.42% for GELU, with the baselines collapsing to 0–8 trades, and additionally reported a 2,834-bar BTC/USDT evaluation and a 1D-CNN/LSTM study. Those results could not be reproduced from the repository: no script, seed list or data file for them was preserved, and the best out-of-sample return obtained by any run in the protocol above is +423%, for *every* activation, with no run anywhere near 0–8 trades. The figures are therefore withdrawn rather than re-tuned. The benchmark in Table I is the complete, runnable replacement, and its verdict is that Q-ALU is competitive with, but not superior to, standard activations on this task.

E. Hardware Latency and Computational Throughput

Benchmarked on a single-threaded x86-64 CPU (Intel Celeron, `torch 2.15.0+cpu`, FP32, $n = 100{,}000$ elements, 200 iterations, best of 3 repeats; `experiments/benchmark.py`). "Backward" is the backward call alone, measured as $t(\text{forward} + \text{backward}) - t(\text{forward})$.

TABLE II
COMPUTATIONAL THROUGHPUT BENCHMARK (100,000 FP32 ELEMENTS, SINGLE THREAD)

Activation	Forward (ms)	Backward (ms)	Forward (M/s)
ReLU	0.334	2.547	299.29
LeakyReLU	1.696	2.661	58.98
SiLU	3.375	4.415	29.63
GELU	2.678	4.234	37.34
FastQALU	**10.384**	**22.600**	**9.63**
Q-ALU 2.0	15.764	20.413	6.34
Q-ALU Ultra	16.427	14.807	6.09
Q-ALU	17.064	17.881	5.86

FastQALU reduces the forward pass latency by **1.64**× relative to transcendental Q-ALU (10.38 ms versus 17.06 ms) and raises forward throughput from 5.86 to 9.63 million elements per second. The end-to-end training step improves by only 1.06× (33.0 ms versus 34.9 ms), because the memory-lean custom autograd function recomputes the rational branch from x in the backward pass instead of retaining intermediates; its backward is consequently 26% *slower* than Q-ALU's eager graph. For inference-only deployment — the relevant regime for HFT signal generation — the 1.64× figure is the operative one.

Note on Hardware Execution: TABLE II evaluates pure PyTorch module implementations on one CPU thread, which isolates algorithmic complexity from hardware-specific graph optimizations. In this setting the per-call Python and allocator overhead dominates at $n = 10^5$ (ReLU's backward, a single elementwise multiply, still costs 2.5 ms), and Q-ALU is 12.1× slower than ReLU end-to-end. The repository ships fused CUDA kernels (`qalu/csrc/qalu_cuda.cu`) that evaluate both branches in a single pass with no Python dispatch, no `torch.where` materialization and no autograd node allocation. We do not report GPU latency figures here: the verification environment had no CUDA device, so the kernels were validated by compilation and numerical equivalence (`experiments/cpp_check.cpp` for the C++ path, syntax and parity checks for the CUDA path) rather than by wall-clock timing.

VII. Practical Deployment and Weight Initialization

For input $X \sim \mathcal{N}(0, 1)$ under default parameters ($\alpha = 1.0, \beta = 1.0, \gamma = 1.5, \delta = 0.5$):

- The output distribution exhibits a positive mean drift: $\mathbb{E}[f(X)] = +0.1976$ (Monte Carlo, 2×10^7 samples; Gauss–Hermite extrapolation gives +0.1973, with quadrature values +0.1994 at $n = 80$ converging monotonically downward).
- The output second moment is $\mathbb{E}[f(X)^2] = 2.6903$, so the output RMS is 1.6402 and the standard deviation is $\sqrt{\mathbb{E}[f^2] - \mathbb{E}[f]^2} = 1.6283$.

Initialization Recommendation: The quantity that enters a Kaiming-style initialization is the reciprocal of the output standard deviation, $\sigma_W = 1/\operatorname{std}(f(X))$, not the RMS: for ReLU, $\operatorname{std}(\text{ReLU}(X)) = 1/\sqrt{2}$, hence the familiar $\sigma_W = \sqrt{2}$. For Q-ALU, $1/1.6283 = 0.6142$. The two conventions must not be mixed — the output RMS of Q-ALU (1.640) is *larger* than that of ReLU (0.707), yet its initialization scale (0.614) is *smaller* than ReLU's (1.414), because Q-ALU's output carries a mean drift and therefore more total second moment per unit of variance.

When training deep networks (> 3 layers) without normalization, weights should therefore be initialized as:

$$W \sim \mathcal{N}\left(0, \sigma^2\right), \quad \text{where } \sigma = \frac{0.6142}{\sqrt{\text{fan_in}}} \tag{28}$$

(The value 0.6097 reported in earlier versions of this work equals $1/\sqrt{\mathbb{E}[f^2]} = 1/1.6402$, i.e. it normalizes the *RMS*

rather than the variance, which leaves each layer with an output variance of $1.6283^2/1.6402^2 = 0.985$ — a 1.5% per-layer shrinkage that compounds to 0.9855^L over L layers. Equation (28) removes this bias; both values are within 1% of each other, so the practical difference is small but the derivation is not.)

In modern deep architectures, pairing Q-ALU directly with standard normalization layers that enforce explicit mean-centering (such as `nn.LayerNorm`) absorbs the asymmetric mean drift ($\mathbb{E}[f(X)] = +0.1976$) while preserving unit variance across arbitrarily deep feedforward networks.

VIII. Conclusion and Future Work

This paper presented **Q-ALU (Quant Asymmetric Leaky Unit)**, an activation function designed to express asymmetric downside risk preferences directly in network calculus: momentum-accelerated tracking on the positive domain, a quadratic noise band followed by an exponential-to-linear cut-loss barrier on the negative domain.

The theoretical results stand on their own: global monotonicity ($f'(x) \geq 0$ for all x), C^0 continuity at the origin, and a bounded *Dynamic Gradient Overshoot* that peaks at 1.1353γ at $x = -2\delta$ and relaxes to the linear asymptote. The implementation contributions also stand: the $0 \times \infty$ autograd hazard is removed, the cancellation-prone exponential is replaced by `expm1`, the rational FastQALU surrogate is accurate to 0.53% and 1.64× faster in the forward pass, and the Python, C++ and CUDA paths agree to finite-difference tolerance.

The empirical claim does not survive. Under a fully reproducible multi regime benchmark with paired seeds, Q-ALU does not outperform ReLU, LeakyReLU, GELU or SiLU on return, Sharpe or Sortino; the predicted turnover collapse does not occur for any activation; and Q-ALU's only measurable signature is a modest reduction in trading frequency accompanied by a deeper drawdown. We therefore withdraw the trading results of the earlier draft (+75.85% versus +5.43% for ReLU) as non-reproducible rather than restate them, and we frame Q-ALU as a principled, numerically robust activation whose behavioural hypotheses about trading performance were tested and, in this setting, not confirmed.

Future work should attack the question the benchmark could not settle: whether asymmetric activations change learning dynamics under the exact algorithm that motivated the design. A PPO agent with a value network, entropy collapse and clip-based credit assignment has failure modes (premature policy determinism) that a contextual-bandit formulation cannot exhibit, so a PPO-specific study with turnover and entropy diagnostics is the natural next experiment. Beyond that, the open questions are whether the overshoot window at $x = -2\delta$ changes behaviour in deeper temporal architectures, and whether the same asymmetry helps in non-financial settings where downside asymmetry is real, such as risk-sensitive control and survival-constrained RL.

References

[1] J. Moody and M. Saffell, "Learning to trade via direct reinforcement," *IEEE Transactions on Neural Networks*, vol. 12, no. 4, pp. 875–889, Jul 2001.

[2] M. F. Dixon, I. Halperin, and P. Bilokon, *Machine Learning in Finance: From Theory to Practice*. Cham, Switzerland: Springer Cham, 2020.

[3] Á. Cartea, S. Jaimungal, and J. Penalva, *Algorithmic and High-Frequency Trading*. Cambridge, UK: Cambridge University Press, 2015.

[4] S. Li, X. Jin, Y. Xuan, X. Zhou, W. Chen, Y.-X. Wang, and X. Yan, "Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 5243–5253. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/67dc7d32e2aab538072d517ff5b1474f-Abstract.html

[5] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th International Conference on Machine Learning (ICML)*, Haifa, Israel, 2010, pp. 807–814.

[6] A. L. Maas, A. Y. Hannun, and A. Y. Ng, "Rectifier nonlinearities improve neural network acoustic models," in *Proceedings of the 30th International Conference on Machine Learning (ICML)*, vol. 30, no. 1, Atlanta, GA, USA, 2013, p. 3.

[7] D. Hendrycks and K. Gimpel, "Gaussian error linear units (gelus)," *arXiv preprint arXiv:1606.08415*, 2016. [Online]. Available: https://arxiv.org/abs/1606.08415

[8] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for activation functions," *arXiv preprint arXiv:1710.05941*, 2017. [Online]. Available: https://arxiv.org/abs/1710.05941

[9] S. Elfwing, E. Uchibe, and K. Doya, "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning," *Neural Networks*, vol. 107, pp. 3–11, Nov 2018.

[10] D. Misra, "Mish: A self regularized non-monotonic activation function," in *Proceedings of the 31st British Machine Vision Conference (BMVC)*, 2020. [Online]. Available: https://arxiv.org/abs/1908.08681

[11] D. Kahneman and A. Tversky, "Prospect theory: An analysis of decision under risk," *Econometrica*, vol. 47, no. 2, pp. 263–291, Mar 1979.

[12] J. Y. Campbell, A. W. Lo, and A. C. MacKinlay, *The Econometrics of Financial Markets*. Princeton, NJ, USA: Princeton University Press, 1997.

[13] R. Roll, "A simple implicit measure of the effective bid-ask spread in an efficient market," *The Journal of Finance*, vol. 39, no. 4, pp. 1127–1139, Sep 1984.

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[15] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, "Efficient backprop," in *Neural Networks: Tricks of the Trade*, ser. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, vol. 1524, pp. 9–50.

[16] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Santiago, Chile, 2015, pp. 1026–1034.

[17] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units (ELUs)," in *International Conference on Learning Representations (ICLR)*, San Juan, Puerto Rico, 2016. [Online]. Available: https://arxiv.org/abs/1511.07289

[18] G. Klambauer, T. Unterthiner, A. Mayr, and S. Hochreiter, "Self-normalizing neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 971–980. [Online]. Available: https://proceedings.neurips.cc/paper/2017/hash/5d44ee6f2c3f71b73125876103c8f6c4-Abstract.html

[19] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International*

Conference on Machine Learning (ICML), Stockholm, Sweden, 2018, pp. 1861–1870.

[20] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb 2015.

[21] F. A. Sortino and L. N. Price, "Performance measurement in a downside risk framework," *The Journal of Investing*, vol. 3, no. 3, pp. 59–64, Fall 1994.

[22] W. K. Newey and J. L. Powell, "Asymmetric least squares estimation and testing," *Econometrica*, vol. 55, no. 4, pp. 819–847, Jul 1987.